

ManicBot

Полный аудит кода, структуры, схем данных, продукта и стратегии продаж

Фокус: архитектура, модули, file map, D1/KV contracts, приоритет улучшений, long-term perspective, features, packaging и go-to-market.

Что проверено	Факт
Worker backend	Прочитаны entrypoints, orchestration, services, tenancy, roles, billing, Google Calendar.
Схемы данных	Сверены D1 schema.sql, Drizzle schema.ts, runtime SQL и docs.
Состояние проекта	`manicbot` tests PASS, `admin-app` build PASS, оба лендинга build PASS.
Продукт	Разобраны owner/admin surfaces, marketing sites, SEO blog, GTM-потенциал.

Executive Summary

- Проект уже выглядит не как один Telegram-бот, а как зачаток vertical SaaS-платформы для beauty-салонов: есть multi-tenant runtime, роли, биллинг, календарь, support, admin surfaces и маркетинговые сайты.
- Главная сильная сторона репозитория — реальное рабочее ядро в `manicbot/src`: там уже есть операционная глубина, backend tests и понятная бизнес-модель вокруг записи, напоминаний и управления салоном.
- Главная слабая сторона — рост шёл быстрее, чем консолидация архитектуры: в коде одновременно живут legacy/KV-паттерны, D1-first-паттерны, параллельные админки, два лендинга, несколько моделей ролей и расходящиеся схемы.
- Перед масштабированием продукта важнее не добавлять ещё одну поверхность, а зафиксировать каноническую модель домена, единый permission layer, единый статусный словарь и единый self-service сценарий для владельца салона.

Methodology

- Пройден весь основной source-слой репозитория: Worker backend, admin-app, два лендинга, blog generator, схемы D1 и ключевые docs.
- Прочитаны реальные entrypoints и самые крупные hotspot-модули: worker.js, handlers/message.js, handlers/callback.js, services/google-calendar-oauth.js, appointments.js, users.js, roles.js, tenant storage/resolver.
- Сверены друг с другом три слоя правды: D1 schema.sql, Drizzle schema.ts в admin-app и фактические SQL/ORM-вызовы в runtime-коде.
- Проверено состояние исполнения: `manicbot` tests проходят, `manicbot-analysis`, `manicbot-landing` и `admin-app` собираются.
- Отдельно оценены продуктовые поверхности, пригодность к продаже, зрелость интерфейсов и долгосрочные точки роста.

Сильные стороны

- Сильное backend-покрытие: 38 test files и 642 теста в Worker-ядре.
- Живой multi-tenant runtime на Cloudflare с webhook, cron, Stripe и Google Calendar.
- Понятный коммерческий скелет: Start / Pro / Studio, feature gating, tenant billing.
- Многоязычность на уровне продукта и контента: RU / UA / EN / PL.
- Уже есть не только бэкэнд, но и marketing/storytelling surface, что важно для продаж.

Ключевые риски

- Schema drift между `manicbot/src/db/schema.sql`, runtime SQL и `manicbot/admin-app/src/server/db/schema.ts`.
- Role drift: backend живёт на `system\_admin` / `tenant\_owner`, admin-app местами живёт на `admin` / `owner`.
- Billing status drift: backend использует `grace\_period`, admin-app — `grace`.
- Timestamp drift: Worker пишет многие значения в миллисекундах, admin-app часто читает и пишет те же поля как секунды.
- Worker перегружен инфраструктурными обязанностями и содержит сайд-эффекты уровня provisioning внутри request path.

- Продуктово не выбран один владелец админской поверхности: HTML admin в Worker, Telegram God Mode и отдельная Next.js mini-app конкурируют между собой.

Карта Репозитория

Пакет	Назначение	Файлы	LOC
manicbot/src	Главное runtime-ядро: Cloudflare Worker, Telegram webhook, Stripe webhook, cron, Google OAuth, HTML admin.	101	13879
manicbot/test	Сильный backend test pack для Worker-логики, роутинга, ролей, биллинга и календаря.	39	5190
manicbot/admin-app/src	Next.js + tRPC + Drizzle консоль управления, по факту ближе к platform-level God Mode.	41	4434
manicbot-analysis/src	Новый презентационный/маркетинговый фронтенд с SEO-блогом и heavy UI-kit базой.	67	6255
manicbot-landing/src	Старый лендинг на React/Vite, который все еще поддерживается как rollback-вариант.	7	504
manicbot-blog	Генератор статического SEO-блога, который встроен в деплой нового лендинга.	1	260

Крупнейшие файлы Worker backend

Файл	LOC
manicbot/src/handlers/callback.js	1337
manicbot/src/handlers/message.js	1222
manicbot/src/services/google-calendar-oauth.js	1013
manicbot/src/worker.js	625
manicbot/src/i18n/ru.js	438
manicbot/src/i18n/pl.js	435
manicbot/src/ai.js	434
manicbot/src/i18n/en.js	434

Крупнейшие файлы admin-app

Файл	LOC
manicbot/admin-app/src/app/tenants/TenantsPageClient.tsx	476

Файл	LOC
manicbot/admin-app/src/app/users/UsersPageClient.tsx	417
manicbot/admin-app/src/app/appointments/AppointmentsPageClient.tsx	311
manicbot/admin-app/src/app/settings/SettingsPageClient.tsx	257
manicbot/admin-app/src/server/api/routers/provisioning.ts	229
manicbot/admin-app/src/app/agents/AgentsPageClient.tsx	223
manicbot/admin-app/src/server/db/schema.ts	176
manicbot/admin-app/src/server/api/routers/users.ts	174

Модульная Структура

Зона	Модуль	Роль
Worker backend	handlers/*	Оркестрация Telegram message/callback/cron потоков.
Worker backend	services/*	Доменные use-case: appointments, users, state, chat, tickets, calendar sync.
Worker backend	tenant/* + roles/* + billing/*	Multi-tenant registry, права, feature gating и Stripe lifecycle.
Worker backend	ui/*	Telegram UI screens, клавиатуры, панели админа и sysadmin.
admin-app	app/* + server/api/* + server/db/*	Platform console, tRPC API, Drizzle ORM schema, Telegram WebApp auth.
manicbot-analysis	components/* + i18n/* + blog integration	Новый лендинг/презентационный слой и SEO-блог.
manicbot-landing	App.jsx + i18n + simple components	Старый маркетинговый лендинг и rollback-вариант Pages.

Основные runtime boundary

Поверхность	Пути	Назначение
Worker edge routes	/webhook, /webhook/:botId, /stripe/webhook, /google/*, /admin*, /setup, /remove-webhook, /calendar/:aptId.ics, /	Единый edge gateway для почти всей системы.
Cron	scheduled() -> handleCron()	Напоминания, billing expiry, calendar resync, cleanup.
admin-app boundary	/api/trpc/[trpc]	Единая backend boundary Next.js mini-app.
Landing delivery	Worker proxy -> Pages project	Новый лендинг и блог живут на Pages, но корень домена завязан на Worker proxy.

Схемы Данных

D1: фактический source of truth для основных business entities

Блок	Таблицы	Назначение
Core tenant data	tenants, bots, tenant_roles, platform_roles, support_agents, tenant_support_agents	Реестр tenants, бот-привязка, роли платформы и tenant-level доступ.
Customer & scheduling	users, masters, appointments, services, tenant_config	Операционные сущности салона: клиенты, мастера, услуги, записи, tenant settings.
Support & comms	local_tickets, human_requests, platform_tickets, platform_ticket_messages	Локальная поддержка салона и платформенная поддержка.
Billing	stripe_customers + поля в tenants	Связка Stripe customer/subscription и статусов подписки.
Google sync (runtime-created)	google_integrations, google_busy_blocks	OAuth-интеграции, busy blocks, watch renewal, двусторонняя синхронизация.

KV: то, что реально осталось важным

Зона	Ключи	Назначение
Ephemeral state	`st:*`, `rl:*`, `chat:*`	State machine, rate limit, AI chat history.
Bot secrets	`bottoken:*`	Bot token-ы остаются в KV, при необходимости в зашифрованном виде.
Global infra	`stripe:evt:*`, `gcal:oauth:*`, `tklock:*`	Idempotency, OAuth sessions, распределённые локи.
Legacy compatibility	tenant-prefixed KV fallback (`b:*`, `t:*`)	Старые пути и graceful degradation для части сценариев.

Schema drift, который надо чинить первым

Drift	Что расходится	Риск
Services ordering	Worker SQL: `sort_order`; admin-app Drizzle: `order`.	Вероятность неконсистентной сортировки и несовместимых миграций.
Support agent type	Worker роли используют `technical`, admin-app пишет `technical_support` в `support_agents.type`.	Часть агентов может не быть видна или не работать одинаково в разных поверхностях.
Platform tickets	Worker пишет `client_chat_id`, `claimed_by`, `sender`, `text`; admin-app schema ожидает `chat_id`, `agent_cid`, `sender_cid`, `body`.	Высокий риск несовместимости при чтении/мутациях тикетов.

Drift	Что расходится	Риск
Tenant roles	SQL schema требует `created_at`, admin-app mutation вставляет role без этого поля.	Дрейф схемы и потенциальные ошибки записи/миграции.
Billing statuses	Backend: `grace_period`; admin-app: `grace`.	Неверная аналитика, фильтры и ручные мутации статусов.
Role vocabulary	Backend опирается на `system_admin` / `tenant_owner`; admin-app местами фильтрует `admin` / `owner`.	Некорректный доступ и неверная сегментация пользователей.
Timestamp units	Worker часто пишет `Date.now()` (ms), admin-app часто пишет/читает `Math.floor(Date.now()/1000)` (sec).	Ломаются даты триала, createdAt, экспорт и аналитика.
Google integration column	Runtime живёт с `google_integration_id`, базовая schema.sql её не содержит и полагается на runtime `ALTER TABLE`.	Скрытая схема затрудняет clean deploy и перенос окружений.

Health Check

Проверка	Статус	Комментарий
Worker backend tests	PASS	642/642 tests, 38 test files. При этом в stderr видны сигналы о хрупкости mock-конфигурации ('KV GET fail', 'undefined/sendMessage').
manicbot-analysis build	PASS	Vite build успешен. Bundle: JS 368.73 kB, CSS 108.65 kB.
manicbot-analysis UI usage	RISK	В `components/ui` лежат 43 UI-компонента, а напрямую продуктом используются только 2: dropdown-menu, toast.
manicbot-landing build	PASS	Legacy-лендинг собирается. Bundle заметно легче: JS 236.43 kB, CSS 17.98 kB.
admin-app typecheck	PASS	TypeScript-проверка проходит.
admin-app production build	PASS	Next.js build успешен, но есть предупреждение о нескольких lockfile и trade-off edge runtime/static generation.
Docs consistency	RISK	README, MULTI_BOT_SETUP и часть setup-docs уже не отражают фактическую D1-first/Next.js картину.

Ключевой вывод по quality layer

- Backend не выглядит мёртвым: тесты сильные, сборки проходят, продуктовый скелет реальный.
- Но quality guardrails не охватывают весь system boundary: особенно admin-app, contract layer между поверхностями и миграционные расхождения.
- Именно поэтому риск здесь не “сломано всё”, а “сложно масштабировать без неожиданных регрессий”.

Топ-15 Улучшений

Блок А. Архитектурное ядро

#	Улучшение	Почему сейчас	Что делать
1	Зафиксировать одну каноническую доменную схему и единый migration contract	Сейчас бизнес-истина размазана между schema.sql, runtime SQL, Drizzle schema и динамическими `ALTER TABLE`.	Свести поли, billing statuses, timestamp units, support types и ticket schema к одному словарю; добавить schema contract tests между Worker и admin-app.
2	Разрезать большие orchestration-модули по use-case потокам	`handlers/callback.js` и `handlers/message.js` уже слишком велики для безопасного роста.	Вынести booking-flow, support-flow, salon-admin-flow, platform-flow и shared AI-actions в отдельные use-case модули.
3	Выбрать одну продуктовую админскую поверхность и построить вокруг неё roadmap	Сейчас есть Worker HTML admin, Telegram God Mode и Next.js admin-app, которые пересекаются по функциям и путают product direction.	Оставить admin-app как platform console, а tenant-owner surface сделать отдельно и планомерно вывести HTML admin в режим legacy/maintenance.
4	Нормализовать модель ролей и разрешений	Разные поверхности говорят о ролях разными словами и проверяют доступ разной логикой.	Ввести один permission matrix, один enum package и обязательную server-side проверку везде.
5	Нормализовать billing lifecycle и временные единицы	Mix `ms`/`sec` и `grace_period`/`grace` создаёт дорогие скрытые баги в биллинге, экспорте и аналитике.	Принять один стандарт времени, один набор статусов и прогнать backfill/migration для существующих данных.

Блок В. Операционная зрелость

#	Улучшение	Почему сейчас	Что делать
6	Довести D1/KV до ясного договора: source of truth vs cache/state	Система уже D1-first, но в коде и документации это ещё не закреплено архитектурно.	Оставить в KV только ephemeral state, locks и secret artifacts; все бизнес-сущности окончательно закрепить за D1.
7	Вынести demo/provisioning side-effects из request path Worker-a	Автопроектирование demo bots и setWebhook внутри обычного fetch path усложняет runtime и операционную предсказуемость.	Перенести в отдельный admin command, script или one-shot job с явным запуском и логированием.
8	Добавить cross-surface integration tests и contract tests	Backend tests сильные, но они почти не ловят расхождения между Worker, admin-app и схемой.	Тестировать role matrix, billing states, ticket schema, tenant reads/writes и экспорт сквозным способом.
9	Сделать observability слоем, а не набором `console.error`	Для SaaS критичны SLA по webhooks, cron, Stripe, Google sync и support ticket routing.	Добавить structured logging, error classes, event names, counters и дашборд по ключевым сбоям.
10	Упростить и похудеть `manicbot-analysis`	Для маркетинговой поверхности пакет содержит 43 UI-компонента, из которых продукт напрямую использует только 2.	Удалить мёртвый UI-kit, сократить bundle и оставить только реально используемые primitive-компоненты.

Блок С. Масштабирование и рост

#	Улучшение	Почему сейчас	Что делать
11	Выстроить единый docs/onboarding набор под текущую архитектуру	Часть документации уже описывает проект, которого в коде больше нет.	Сделать актуальные docs для runtime, tenant onboarding, bot provisioning, billing, admin-app и delivery topology.
12	Ввести централизованный event/audit trail	Для поддержки, биллинга, ролей и ручных действий нужна прозрачная история “кто что сделал и когда”.	Собирать audit events для role grants, billing changes, appointment status transitions, manual cancellations и provisioning.
13	Усилить security hygiene и operator model	В коде и конфигурации видны hardcoded operator assumptions (`CREATOR_ID`, `ADMIN_CHAT_ID`, локальные secret artifacts).	Убрать жёсткие ID из кода, перевести всё в безопасный operator config и задокументировать процедуру ротации.
14	Разнести release boundaries по продуктовым пакетам	Сейчас в одном репозитории живут core runtime, platform console, два лендинга и блог; изменения трудно выпускать независимо.	Минимум — формализовать ownership и release notes по пакетам; дальше — workspace conventions и semantic boundaries.
15	Сделать product analytics first-class слоем	Чтобы продавать и улучшать продукт, нужно видеть не только runtime health, но и conversion, no-show, retention, usage of features.	Собирать воронку от первого сообщения до подтверждённой записи, повторные визиты, активность мастеров и апсейл в платные планы.

Долгосрочная Перспектива

- Нужно выбрать identity продукта: это “AI-бот для записи” или “операционная платформа для салонов красоты”. Второй вариант сильнее и защищённее, но требует консолидации owner-facing UX и аналитики.
- Telegram как входной канал хорош для старта и польского рынка, но долгосрочно стоит считать его только первым каналом, а не единственной дверью в продукт.
- При масштабировании появятся вопросы GDPR, управления персональными данными, согласий, экспортов, удаления истории и tenant-level auditability.
- Если целиться в сети/франшизы, нужно заранее думать о multi-location модели: не просто tenant, а группа салонов, несколько календарей, централизованные роли, единые отчёты.
- AI-слой должен стать controllable feature: стоимость, fallback-политика, guardrails, объяснимость действий и ручной override для салона.
- Дальняя ценность продукта лежит не в одном бронировании, а в повторных визитах, загрузке мастеров, no-show control и revenue automation.

Топ-10 идей для продукта

Идея	Бизнес-смысл
Депозиты и частичная предоплата	Снижает no-show и сразу повышает monetization power продукта.
Waitlist + авто-подбор освободившегося слота	Даёт салону дополнительную выручку без ручной координации.
Реактивационные кампании по неактивным клиентам	Поднимает LTV, а не только acquisition.
Пакеты услуг и membership logic	Делает продукт ближе к revenue OS, а не только booking tool.
Автоматический сбор отзывов после визита	Создаёт growth loop и доверие для новых клиентов.
Referral engine / “приведи подругу”	Дешёвый и органичный канал роста для beauty vertical.
Умный рескейджул и no-show scoring	Повышает заполняемость и качество расписания.
Единая inbox-панель по каналам	Готовит продукт к выходу за пределы одного Telegram-канала.
Owner analytics dashboard	Даёт основание платить не за чат-бота, а за управленческую систему.
Multi-location / franchise mode	Открывает более дорогой сегмент и повышает ARPU.

Что стоит улучшить на сайте и в упаковке

- На сайте должен продаваться не “AI”, а измеримый бизнес-результат: меньше неявок, меньше ручной переписки, больше записей вне рабочего времени.
- Нужен живой demo flow с одним-двумя реальными сценариями: запись, перенос, напоминание, жалоба/поддержка.
- Нужен простой ROI-калькулятор: сколько заявок теряется без автоответа, сколько часов администратора экономится, какой эффект даёт снижение no-show.

- Нужна отдельная страница для owner value: роли, контроль мастеров, графики, аналитика, биллинг, интеграции.
- Нужен блок доверия: кейсы, скриншоты реального бота, доказательства многоязычности, интеграция с Google Calendar, прозрачные тарифы.

Стратегия Продаж

- ICP на ближайший горизонт: небольшие студии и салоны на 2-5 мастеров в Польше. У них уже есть боль координации, но ещё нет сложного enterprise procurement.
- Позиционирование: не “ещё один бот”, а “система записи и загрузки салона в мессенджере, которая работает 24/7 и уменьшает no-show”.
- Entry offer: быстрый запуск за 3-7 дней с подключением бота, услуг, мастеров, календаря и готовым demo-script для owner.
- Коммерчески имеет смысл добавить setup/onboarding fee поверх recurring plan. Это снижает барьер продаж как “услуги с результатом”, а не только как софт.
- Основные каналы первых продаж: founder-led outreach в Telegram/Instagram, локальные beauty-сообщества, партнёры по маркетингу/сайтам для салонов, сарафан через действующих мастеров.
- Продажа должна идти от ROI: время ответа, количество записей вне рабочего времени, сокращение пустых окон, количество перенесённых/спасённых визитов.
- Апсейл-логика: Start -> Pro через календарь, поддержку и аналитику; Pro -> Studio через white-label, multi-location и advanced automation.
- Отдельно нужен partner motion для агентств и консультантов, которые настраивают digital stack салонам. Это дешёвый канал масштабирования без большого sales team.

Roadmap

Горизонт	Фокус
0-3 месяца	Зафиксировать каноническую схему, role/billing словари, timestamp units, убрать критический drift, разрезать большие handlers.
3-6 месяцев	Сделать чёткий owner-facing кабинет, нормальную аналитику, audit trail и productized onboarding.
6-12 месяцев	Добавить revenue-фичи: депозиты, waitlist, реактивации, отзывы, membership logic, customer segmentation.
12+ месяцев	Выход в multi-channel, franchise/multi-location mode, партнёрский канал продаж и более дорогие B2B-пакеты.

Общее резюме

ManicBot уже перерос стадию “интересного pet-project бота” и имеет реальную основу для vertical SaaS в beauty. Код показывает, что главная ценность уже есть: multi-tenant runtime, биллинг, календарь, роли, поддержка, языки и живая delivery-модель.

Следующий правильный шаг — не расползаться ещё шире, а сконцентрироваться на консолидации модели данных, прав и owner-facing продукта. Если это сделать, проект можно продавать уже не как автоматизацию чата, а как систему операционного управления записью и загрузкой салона.

Приложение: Trees

[manicbot/src](#)

```

■■■ .DS_Store
■■■ admin
■   ■■■ provisioning.js
■   ■■■ seed.js
■■■ ai.js
■■■ billing
■   ■■■ config.js
■   ■■■ features.js
■   ■■■ lifecycle.js
■   ■■■ storage.js
■   ■■■ stripe.js
■   ■■■ webhooks.js
■■■ config.js
■■■ db
■   ■■■ schema.sql
■■■ handlers
■   ■■■ callback.js
■   ■■■ cron.js
■   ■■■ message.js
■■■ i18n
■   ■■■ en
■   ■   ■■■ admin.js
■   ■   ■■■ billing.js
■   ■   ■■■ booking.js
■   ■   ■■■ gcal.js
■   ■   ■■■ general.js
■   ■   ■■■ index.js
■   ■   ■■■ master.js
■   ■   ■■■ menu.js
■   ■   ■■■ meta.js
■   ■   ■■■ screens.js
■   ■   ■■■ services.js
■   ■   ■■■ support.js
■   ■   ■■■ sysadmin.js
■   ■■■ en.js
■   ■■■ index.js
■   ■■■ pl
■   ■   ■■■ admin.js
■   ■   ■■■ billing.js
■   ■   ■■■ booking.js
■   ■   ■■■ gcal.js
■   ■   ■■■ general.js
■   ■   ■■■ index.js
■   ■   ■■■ master.js
■   ■   ■■■ menu.js
■   ■   ■■■ meta.js
■   ■   ■■■ screens.js
■   ■   ■■■ services.js
■   ■   ■■■ support.js
■   ■   ■■■ sysadmin.js
■   ■■■ pl.js
■   ■■■ ru
■   ■   ■■■ admin.js
■   ■   ■■■ billing.js
■   ■   ■■■ booking.js
■   ■   ■■■ gcal.js
■   ■   ■■■ general.js
■   ■   ■■■ index.js

```


manicbot/src (part 2)

```
■ ■ ■■■ master.js
■ ■ ■■■ menu.js
■ ■ ■■■ meta.js
■ ■ ■■■ screens.js
■ ■ ■■■ services.js
■ ■ ■■■ support.js
■ ■ ■■■ sysadmin.js
■ ■■■ ru.js
■ ■■■ ua
■ ■ ■■■ admin.js
■ ■ ■■■ billing.js
■ ■ ■■■ booking.js
■ ■ ■■■ gcal.js
■ ■ ■■■ general.js
■ ■ ■■■ index.js
■ ■ ■■■ master.js
■ ■ ■■■ menu.js
■ ■ ■■■ meta.js
■ ■ ■■■ screens.js
■ ■ ■■■ services.js
■ ■ ■■■ support.js
■ ■ ■■■ sysadmin.js
■ ■■■ ua.js
■■■ i18n.js
■■■ notifications.js
■■■ patterns.js
■■■ roles
■ ■■■ roles.js
■■■ services
■ ■■■ appointments.js
■ ■■■ calendar.js
■ ■■■ chat.js
■ ■■■ google-calendar-oauth.js
■ ■■■ services.js
■ ■■■ state.js
■ ■■■ tickets.js
■ ■■■ users.js
■■■ support
■ ■■■ tickets.js
■■■ telegram.js
■■■ tenant
■ ■■■ migration.js
■ ■■■ resolver.js
■ ■■■ storage.js
■■■ ui
■ ■■■ admin.js
■ ■■■ billing.js
■ ■■■ booking.js
■ ■■■ keyboards.js
■ ■■■ screens.js
■ ■■■ sysadmin.js
■■■ utils
■ ■■■ date.js
■ ■■■ db.js
■ ■■■ helpers.js
■ ■■■ ics.js
■ ■■■ kv.js
■ ■■■ landing-pages-proxy.js
```

manicbot/src (part 3)

```
■ ■■■ security.js
■■■ worker.js
```

manicbot/admin-app/src

```

■■■ app
■   ■■■ _components
■   ■■■ agents
■     ■■■ AgentsPageClient.tsx
■     ■■■ page.tsx
■   ■■■ api
■     ■■■ trpc
■       ■■■ [trpc]
■       ■■■ route.ts
■   ■■■ appointments
■     ■■■ AppointmentsPageClient.tsx
■     ■■■ page.tsx
■   ■■■ billing
■     ■■■ BillingPageClient.tsx
■     ■■■ page.tsx
■   ■■■ DashboardClient.tsx
■   ■■■ layout.tsx
■   ■■■ page.tsx
■   ■■■ settings
■     ■■■ page.tsx
■     ■■■ SettingsPageClient.tsx
■   ■■■ stripe
■     ■■■ page.tsx
■   ■■■ system
■     ■■■ page.tsx
■     ■■■ SystemPageClient.tsx
■   ■■■ tenants
■     ■■■ page.tsx
■     ■■■ TenantsPageClient.tsx
■   ■■■ users
■     ■■■ page.tsx
■     ■■■ UsersPageClient.tsx
■■■ components
■   ■■■ dashboard
■     ■■■ OverviewChart.tsx
■   ■■■ layout
■     ■■■ Shell.tsx
■     ■■■ TelegramGate.tsx
■   ■■■ ui
■■■ env.js
■■■ server
■   ■■■ api
■     ■■■ root.ts
■     ■■■ routers
■       ■■■ appointments.ts
■       ■■■ billing.ts
■       ■■■ export.ts
■       ■■■ metrics.ts
■       ■■■ provisioning.ts
■       ■■■ settings.ts
■       ■■■ stripe.ts
■       ■■■ system.ts
■       ■■■ tenants.ts
■       ■■■ users.ts
■     ■■■ trpc.ts
■   ■■■ auth
■     ■■■ telegram.ts
■   ■■■ db

```

manicbot/admin-app/src (part 2)

```
■      ■■■ index.ts
■      ■■■ schema.ts
■■■ styles
■      ■■■ globals.css
■■■ trpc
      ■■■ query-client.ts
      ■■■ react.tsx
      ■■■ server.ts
```

manicbot-analysis/src

```
■■■ App.tsx
■■■ assets
■   ■■■ hero.png
■   ■■■ manicbot-emoji-mark-ui.png
■   ■■■ manicbot-emoji-mark.png
■   ■■■ react.svg
■   ■■■ vite.svg
■■■ components
■   ■■■ CtaSection.tsx
■   ■■■ FaqSection.tsx
■   ■■■ FeaturesSection.tsx
■   ■■■ Footer.tsx
■   ■■■ Header.tsx
■   ■■■ HeroSection.tsx
■   ■■■ HowSection.tsx
■   ■■■ LanguageSwitcher.tsx
■   ■■■ PricingSection.tsx
■   ■■■ SeoHead.tsx
■   ■■■ TelegramPhoneDemo.tsx
■   ■■■ TestimonialsSection.tsx
■   ■■■ ThemeToggle.tsx
■   ■■■ ui
■       ■■■ accordion.tsx
■       ■■■ alert.tsx
■       ■■■ aspect-ratio.tsx
■       ■■■ avatar.tsx
■       ■■■ badge.tsx
■       ■■■ breadcrumb.tsx
■       ■■■ button.tsx
■       ■■■ calendar.tsx
■       ■■■ card.tsx
■       ■■■ carousel.tsx
■       ■■■ checkbox.tsx
■       ■■■ collapsible.tsx
■       ■■■ command.tsx
■       ■■■ context-menu.tsx
■       ■■■ dialog.tsx
■       ■■■ drawer.tsx
■       ■■■ dropdown-menu.tsx
■       ■■■ form.tsx
■       ■■■ hover-card.tsx
■       ■■■ input.tsx
■       ■■■ label.tsx
■       ■■■ menubar.tsx
■       ■■■ navigation-menu.tsx
■       ■■■ popover.tsx
■       ■■■ progress.tsx
■       ■■■ radio-group.tsx
■       ■■■ resizable.tsx
■       ■■■ scroll-area.tsx
■       ■■■ select.tsx
■       ■■■ separator.tsx
■       ■■■ sheet.tsx
■       ■■■ skeleton.tsx
■       ■■■ slider.tsx
■       ■■■ sonner.tsx
■       ■■■ switch.tsx
■       ■■■ table.tsx
```

manicbot-analysis/src (part 2)

```
■      ■■■ tabs.tsx
■      ■■■ textarea.tsx
■      ■■■ toast.tsx
■      ■■■ toaster.tsx
■      ■■■ toggle-group.tsx
■      ■■■ toggle.tsx
■      ■■■ tooltip.tsx
■■■ constants.ts
■■■ hooks
■      ■■■ use-toast.ts
■■■ i18n
■      ■■■ en.ts
■      ■■■ index.ts
■      ■■■ pl.ts
■      ■■■ ru.ts
■      ■■■ ua.ts
■■■ index.css
■■■ lib
■      ■■■ utils.ts
■■■ main.tsx
■■■ theme
    ■■■ ThemeProvider.tsx
```

manicbot-landing/src

```
■■■ App.css
■■■ App.jsx
■■■ Carousel.css
■■■ Carousel.jsx
■■■ ChatMockup.css
■■■ ChatMockup.jsx
■■■ i18n
■      ■■■ index.js
■      ■■■ locales
■          ■■■ en
■              ■■■ common.json
■          ■■■ pl
■              ■■■ common.json
■          ■■■ ru
■              ■■■ common.json
■          ■■■ uk
■              ■■■ common.json
■■■ index.css
■■■ main.jsx
■■■ test
    ■■■ App.test.jsx
    ■■■ setup.js
```